Dynamic Host Blocking System using SDN (Mininet + POX)

1. Problem Understanding & Setup

Problem: Detect suspicious network traffic and dynamically block malicious hosts using Software Defined Networking (SDN).

Objective:

- Monitor network traffic
- Detect suspicious hosts based on packet behavior
- Dynamically install blocking rules
- Verify rule installation in the switch
- Log events for monitoring

Topology:

```
h1   h2   h3
 \   |   /
    s1
     |
  Controller
```

Roles:

h1 → normal client

h2 → server

h3 → attacker/suspicious host

s1 → OpenFlow switch

Controller → POX SDN controller

Tools Used:

Mininet – network emulator

POX – SDN controller

OpenFlow – protocol used to control switches

2. SDN Logic & Flow Rule Implementation

The controller listens to PacketIn events from the switch.

Flow of packets:

Host → Switch → Controller

Detection Logic:

The controller counts packets sent by each host.

Example logic:

```
if packet_count[src_ip] > THRESHOLD:
    block host
```

Threshold used in project: 20 packets.

Flow Rule Installation:

Example OpenFlow rule installed in switch:

```
match: ip,nw_src = 10.0.0.3
action: DROP
```

priority: 100

This blocks traffic from the suspicious host.

Controller Behavior:

1. Forward normal packets

2. Detect suspicious traffic

3. Install blocking rule in switch

3. Functional Correctness (Demo)

Forwarding Test:

Command:

pingall

Expected result:

0% packet loss

Monitoring:

Controller logs packet counts.

Example:

Traffic from 10.0.0.3 count=21

Blocking:

When threshold exceeded:

ALERT: Suspicious host detected

Blocking host 10.0.0.3

Verification:

Command:

dpctl dump-flows

Example Output:

ip,nw_src=10.0.0.3 actions=drop

4. Performance Observation & Analysis

Latency Measurement:

Command:

h1 ping h2

Measures round-trip time.

Throughput Measurement:

Commands:

h2 iperf -s

h1 iperf -c h2

Measures bandwidth between hosts.

Flow Table Changes:

Before attack → normal forwarding rules

After attack → nw_src=10.0.0.3 actions=drop

Packet Statistics:

Controller tracks packet counts:

Traffic from 10.0.0.3 count=21

5. Explanation, Viva & Validation

What is SDN?

Software Defined Networking separates the control plane from the data plane.

Switch → data plane

Controller → control plane

Role of Controller:

• Monitor traffic

• Analyze packets

• Install flow rules

• Control switch behavior

OpenFlow:

Allows controller to program the switch flow table.

Flow Rule Components:

match → condition

action → operation

priority → rule order

Example:

match: source IP

action: drop

Validation Steps:

1. Normal traffic test:

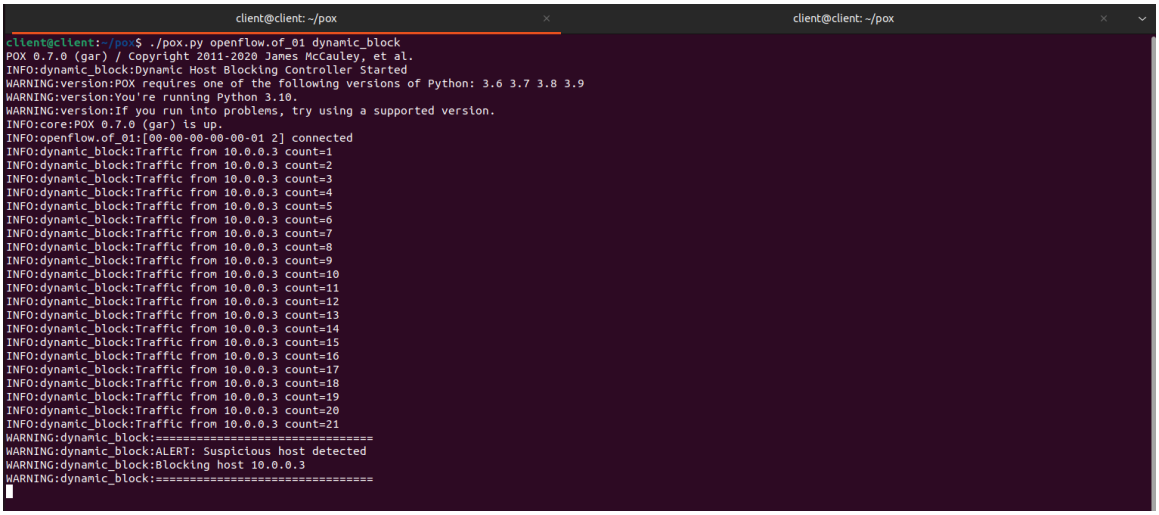
pingall

2. Attack simulation:

h3 ping h1

3. Flow rule verification:

dpctl dump-flows



```
client@client: ~/pox                                client@client: ~/pox
client@client:~/pox$ ./pox.py openflow.of_01 dynamic_block
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:dynamic_block:Dynamic Host Blocking Controller Started
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.10.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:dynamic_block:Traffic from 10.0.0.3 count=1
INFO:dynamic_block:Traffic from 10.0.0.3 count=2
INFO:dynamic_block:Traffic from 10.0.0.3 count=3
INFO:dynamic_block:Traffic from 10.0.0.3 count=4
INFO:dynamic_block:Traffic from 10.0.0.3 count=5
INFO:dynamic_block:Traffic from 10.0.0.3 count=6
INFO:dynamic_block:Traffic from 10.0.0.3 count=7
INFO:dynamic_block:Traffic from 10.0.0.3 count=8
INFO:dynamic_block:Traffic from 10.0.0.3 count=9
INFO:dynamic_block:Traffic from 10.0.0.3 count=10
INFO:dynamic_block:Traffic from 10.0.0.3 count=11
INFO:dynamic_block:Traffic from 10.0.0.3 count=12
INFO:dynamic_block:Traffic from 10.0.0.3 count=13
INFO:dynamic_block:Traffic from 10.0.0.3 count=14
INFO:dynamic_block:Traffic from 10.0.0.3 count=15
INFO:dynamic_block:Traffic from 10.0.0.3 count=16
INFO:dynamic_block:Traffic from 10.0.0.3 count=17
INFO:dynamic_block:Traffic from 10.0.0.3 count=18
INFO:dynamic_block:Traffic from 10.0.0.3 count=19
INFO:dynamic_block:Traffic from 10.0.0.3 count=20
INFO:dynamic_block:Traffic from 10.0.0.3 count=21
WARNING:dynamic_block:================================
WARNING:dynamic_block:ALERT: Suspicious host detected
WARNING:dynamic_block:Blocking host 10.0.0.3
WARNING:dynamic_block:================================
```

client@client: ~/pox client@client: ~/pox

```
client@client:~/pox$ sudo mn --topo single,3 --controller=remote,ip=127.0.0.1,port=6633
[sudo] password for client:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> h3 ping h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.43 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1.74 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1.61 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=1.52 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=1.52 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=1.54 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=1.44 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=1.34 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=1.83 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=4.52 ms
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=1.46 ms
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=1.74 ms
64 bytes from 10.0.0.1: icmp_seq=13 ttl=64 time=1.62 ms
64 bytes from 10.0.0.1: icmp_seq=14 ttl=64 time=1.26 ms
64 bytes from 10.0.0.1: icmp_seq=15 ttl=64 time=1.30 ms
64 bytes from 10.0.0.1: icmp_seq=16 ttl=64 time=1.86 ms
^C
--- 10.0.0.1 ping statistics ---
30 packets transmitted, 16 received, 46.6667% packet loss, time 29325ms
rtt min/avg/max/mdev = 1.260/1.731/4.520/0.740 ms
mininet> dpctl dump-flows
*** s1 ------------------------------------------------------------------------
 cookie=0x0, duration=31.163s, table=0, n_packets=14, n_bytes=1372, priority=100,ip,nw_src=10.0.0.3 actions=drop
mininet>
```